

category: networking-protocol

tags: [zkteco, pull-protocol, c3, tcp, firmware, ack-error, handshake]

severity: high

created: 2026-07-29

updated: 2026-07-29

project-origin: iAccess.Devices.ZktecoPull (App-Access-V2)

ZKTeco C3 firmware LUÔN trả ACK_ERROR cho CMD_CONNECT dù kết nối thành công

Tình huống gặp phải

Port giao thức ZKTeco PULL/C3 (TCP 4370, frame `0xAA...0x55`) sang .NET. Tầng transport

gửi `CMD_CONNECT (0x76)` payload `[FE FE FE FE]`, thiết bị thật trả `CMD_ACK_ERROR (0xC9)`.

Giả thuyết ban đầu: thiếu comm-password. Sau khi USER xác nhận thiết bị không đặt password, cần điều tra nguyên nhân khác.

Triệu chứng / Lỗi

```
PullConnectException: Handshake thất bại: thiết bị trả CommandId=0xC9  
(0xC8=ACK_OK, 0xC9=ACK_ERROR). IP=192.168.1.203:4370
```

Thiết bị vẫn accept TCP connection (port OPEN). Chỉ `CMD_CONNECT` bị `ACK_ERROR`.

Nguyên nhân gốc rễ (Root Cause)

****Firmware ZKTeco C3 (một số model) LUÔN trả `ACK_ERROR (0xC9)` cho `CMD_CONNECT`,**

kể cả khi kết nối thành công, và KÈM session ID trong payload.**

Hex dump thực nghiệm (4/4 variant — bất kể MachineId hay payload):

```
Sent 12B: AA-01-76-04-00-FE-FE-FE-FE-46-77-55 (CMD_CONNECT)  
Recv 9B: AA-01-C9-01-00-F3-13-D9-55 (ACK_ERROR + sessionId=[F3])
```

`payloadLen=1, payload=[F3]` = session ID hợp lệ. Device connect thành công nhưng

dùng `ACK_ERROR` thay vì `ACK_OK` làm code trả lời cho `CMD_CONNECT`.

ZKTecoCore gốc biết điều này — ZKTecoClient.cs dòng 135 chấp nhận **cả hai** có chủ ý:

```
if (response.CommandId == CMD_ACK_OK || response.CommandId == CMD_ACK_ERROR)  
{  
    _sessionId = response.Payload; // session ID có mặt trong payload dù là ACK_ERROR  
    LoadDeviceSchema();  
    return true; // connected!  
}
```

Giải pháp

Trong `ConnectAsync()`, accept cả `ACK_OK` lẫn `ACK_ERROR` cho riêng **handshake CONNECT**.

Với mọi lệnh khác (`SETDATA`, `CONTROL`,...) qua `SendCommandAsync`, `ACK_ERROR` vẫn là thất bại.

```
// PullTcpTransport.ConnectAsync() - ĐÚNG
bool connectAccepted = response.CommandId == PullConstants.CMD_ACK_OK
                        || response.CommandId == PullConstants.CMD_ACK_ERROR;
if (connectAccepted)
{
    _connected = true;
    _logger.LogInformation(
        "ZktecoPull Connected: ResponseCmd=0x{Cmd:X2} SessionIdLen={Len}",
        response.CommandId, response.Payload.Length);
    return response.Payload; // SessionId - có trong payload dù ACK_ERROR
}
throw new PullConnectException($"Handshake thất bại:
CommandId=0x{response.CommandId:X2}");
```

Áp dụng lại (How to reuse)

- Khi thấy ZKTeco C3 device reject `CMD_CONNECT` với `ACK_ERROR` → KHÔNG đặt password → trước tiên kiểm tra hex dump: nếu payload của `ACK_ERROR` có dữ liệu (sessionId) thì đây là firmware behavior, chấp nhận cả hai.
- Khi port ZKTecoCore sang .NET và thấy "bug" coi `ACK_ERROR` là thành công → đây là intentional design, KHÔNG phải bug. Chỉ "fix" này cho các lệnh thông thường (data commands), KHÔNG fix cho `CONNECT` handshake.
- Khi viết transport layer cho giao thức binary mới: luôn log HEX DUMP đầy đủ (sent + received) ở mức Debug/Info cho `CONNECT` handshake để debug dễ hơn.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Device trả CÙNG response bất kể `MachineId` (0x00 hay 0x01) và bất kể payload ([FE FE FE FE] hay rỗng) — điều này có nghĩa `MachineId` và payload không quan trọng với firmware này trong giai đoạn `CONNECT`.
- ⚠ Session ID trong `ACK_ERROR` payload có thể ngắn hơn bình thường (1 byte [F3] thay vì 4 byte). Code downstream phải xử lý session ID độ dài bất kỳ.
- ⚠ Không phải mọi ZKTeco model đều có behavior này — có model trả `ACK_OK` bình thường. ZKTecoCore chấp nhận cả hai chính là để handle cả hai trường hợp.
- ⚠ Nếu future fix "chuẩn hóa" `ConnectAsync` chỉ accept `ACK_OK` → PHẢI verify lại với thiết bị thật, không chỉ với mock server.

Tham chiếu

- ZKTecoCore/ZKTecoClient.cs dòng 135 (reference code, CHỈ ĐỌC)
- BUG-073: [docs/bugs/BUG-073-zkpull-connect-ack-error-password.md](#)
- STEP-6.2: [docs/plans/PLAN-devicehost-zktecopull-2026-07-28/steps/STEP-6.2-fix-connect-issue.md](#)

- Fix:
`iAccessDesktopv2.Avalonia/iAccess.Devices.ZktecoPull/Transport/PullTcpTransport.cs — ConnectAsync()`